

DSA STUDY BLUEPRINT SERIES

119. Advanced Graph Concepts

Advanced Graph Architectures and Cycle Traversals

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **DFS Spanning Forest:** DFS splits nodes into spanning tree branches.
- Tracks back-edges to detect cycles.

Memory & Execution Blueprint

Back-edge points from node 4 to ancestor 1 $\Rightarrow$ cycle confirmed.

C++ Implementation Reference

Standard code layout and syntax

```
// Advanced Connectivity & DFS properties  
// Low-link and discovery timestamp variables resolve articulation nodes.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
DFS Tree Scan	$O(V + E)$ time, $O(V)$ space

Key Practices & Gotchas

- **Important:** Understanding back-edges is key to cycle finding algorithms.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.